Assignment: MapReduce Questions

Commands / Steps:

- To create workspace:
    1. new java project and then all checkmark like: public and main method
    2. import jar files
    3. create class file
    4. write code
    5. export and make JAR file of it
    6. create directory in haddop: command - hdfs dfs -mkdir /<>
    7. create input folder and directory
    8. run command - $ hadoop jar /home/sois/eclipse-workspace/wcStu.jar stuAssignment.stuCount /stuAss/input/student_dataset.txt /stuAss/output

- Import: in eclips we hacve to import .jar file, the name of jar file is:
    - hadoop-mapreduce-clinet-core.jar 3.2.1 size - 1.7MB (Inside opt -> Hadoop - share - Hadoop -> mapreduce)
    - haddop-common-jar 3.2.1.jar and size : 4.1MB ((Inside opt -> Hadoop - share - Hadoop -> common)

- Other Commands:
    1. To start Hadoop and yarn: start-dfs.sh && start-yarn.sh
    2. to copy command from local to Hadoop: hdfs dfs -put <local-filepath> <directory>

    3. command to extract output: hadoop jar wc.jar <packagename>.<classname> <input-dir> <output-dir>

    Ex: hadoop jar /home/sois/eclipse-workspace/wcFour.jar coutLenghtFour.coutLenghtFour /assFour/input/four.txt /assFour/output

    4. to copy from Hadoop to local folder: $ hdfs dfs -get /logAsi /home/sois/Desktop/
    5. To stop Hadoop and yarn: stop-dfs.sh && stop-yarn.sh

1. Find the count of each word from a given input file.

Code:

```
public class wordCount {

    public static class testWordCountMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
        public void map(LongWritable key, Text value, Context context) throws IOException,
    InterruptedException {
                String line = value.toString();
                StringTokenizer tokenizer = new StringTokenizer (line);

                while (tokenizer.hasMoreTokens() ) {
                    value.set(tokenizer.nextToken() );
                    context.write(value, new IntWritable(1));
                }
        }
    }
    public static class testWordCountReducer extends Reducer <Text, IntWritable, Text, IntWritable > {
        public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {
                int sum = 0;
                for (IntWritable x: values) {
                    sum += x.get();
                }
                context.write(key, new IntWritable(sum) );
        }
    }
    public static void main(String[] args) throws Exception{
        // TODO Auto-generated method stub
        Configuration conf = new Configuration();

        Job job = Job.getInstance(conf, "my count");

        job.setJarByClass(wordCount.class);
        job.setMapperClass(testWordCountMapper.class);
        job.setReducerClass(testWordCountReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));

        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0: 1);
    }
}
```

2. Create dataset with fields like 'Student Name', 'Institute', 'Program Name', and 'Gender' and solve following questions.

a. Compute number of students from each Institute.

b. Number of students enrolled to any program.

c. Number of 'boy' and 'girl' students.

d. Number of 'boy' and 'girl' students from selected Institute.

Code:

```
package stuAssignment;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import java.io.IOException;
import java.util.*;

public class stuCount {
    public static class stuMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
            public final static IntWritable one = new IntWritable(1);
            private Text outKey = new Text();

            public void map(LongWritable key, Text value, Context context) throws IOException,
InterruptedException {

                    String line = value.toString();
                    if (line.startsWith("Student Name")) return; // it will going to skip the header

                    String[] fields = line.split(",");
                    if(fields.length < 4) return;

                    String ins = fields[1].trim();
                    String pro = fields[2].trim();
                    String gen = fields[3].trim();

                    //a. count per institute
                    outKey.set("Institute - " + ins);
                    context.write(outKey, one);
```

```
                        //b. total students enrolled to any program
                        if(!pro.isEmpty()) {
                                outKey.set("Enrolled Student - ");
                                context.write(outKey, one);
                        }

                        //c. count per gender
                        outKey.set("Gender - " + gen);
                        context.write(outKey, one);

                        //d. count per institute+gender
                        outKey.set("Institute and Gender - " + ins + " -  " + gen);
                        context.write(outKey, one);
                }
        }
        public static class stuReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
                public void reduce(Text key, Iterable<IntWritable> values,Context context) throws
IOException, InterruptedException {

                        int sum = 0;
                        for (IntWritable val : values) {
                            sum += val.get();
                        }
                  context.write(key, new IntWritable(sum));
                        }
        }

        public static void main(String[] args) throws Exception {
                Configuration conf = new Configuration();

         Job job = Job.getInstance(conf, "Student Analysis");

         job.setJarByClass(stuCount.class);
         job.setMapperClass(stuMapper.class);
         job.setReducerClass(stuReducer.class);
         job.setOutputKeyClass(Text.class);
         job.setOutputValueClass(IntWritable.class);
         FileInputFormat.addInputPath(job, new Path(args[0]));
         FileOutputFormat.setOutputPath(job, new Path(args[1]));
         System.exit(job.waitForCompletion(true) ? 0 : 1);
        }
}
```

3. **Dataset: Temperature of Indian Cities. Fields of dataset are Date, Average Temperature, City, Country, Latitude and Longitude (Dataset is attached). Solve following questions**

    a. Find maximum and minimum temperature of all cities from the given dataset

    b. Count number of data point for each city.

    c. Find the maximum and minimum temperature for city **Bangalore** from the given dataset.

    d. Find the maximum and minimum temperature for any given city from the given dataset. City name should be passed through command line argument.

Code:

```
package tempCount;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import java.io.IOException;
import java.util.*;

  public class tempCount {
    public static class tempMapper extends Mapper<Object, Text, Text, DoubleWritable> {

      private Text outKey = new Text();

      private DoubleWritable temp = new DoubleWritable();

      private String tarCity = null;

      protected void setup(Context context)
      {
        tarCity = context.getConfiguration().get("filter.city",null);
      }
```

```
            public void map(Object key, Text value, Context context) throws IOException,
        InterruptedException {

            String line = value.toString().trim();

            if (line.isEmpty()) return; // it will going to skip the header

            //Field: Data avgTemprature city country latitude longitude
            String[] fields = line.split("\\s+");
            if(fields.length < 3) return;

            String dateStr = fields[0];
            String cityName = fields[2];

            //skip header row if present
            if(dateStr.equalsIgnoreCase("Date")) return;

            try {
               double tempp = Double.parseDouble(fields[1]);
                  if(tarCity != null && !cityName.equalsIgnoreCase(tarCity)) {
                  return;
                  }
               outKey.set(cityName);
               temp.set(tempp);
               context.write(outKey, temp);
            }catch(NumberFormatException e)
            {
            }
        }
    }

    public static class tempReducer extends Reducer<Text, DoubleWritable, Text, Text> {
       public void reduce(Text key, Iterable<DoubleWritable> values,Context context) throws
IOException, InterruptedException {
            double max = Double.NEGATIVE_INFINITY;
            double min = Double.POSITIVE_INFINITY;
            long count = 0;

            for (DoubleWritable val: values)
            {
               double v = val.get();
               if(v>max) max =v;
               if(v<max) min =v;
               count++;
            }
            String result = String.format("MaxTemp = %.3f MinTemp= %.3f DataPoints =
%d",max,min,count);
            context.write(key, new Text(result));
        }
    }
```

```
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Student Analysis");

        job.setJarByClass(tempCount.class);

        job.setMapperClass(tempMapper.class);

        job.setReducerClass(tempReducer.class);

        job.setOutputKeyClass(Text.class);

        job.setOutputValueClass(DoubleWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));

        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);

    }

}
```

4. **Count the number of words based of character length in a given file.**

Example:

Hello world hope all is well Is BDA program boring

Expected output should be:

5	3
4	2
3	2
2	2
7	1

Code:

```
import java.io.IOException;
import java.util.*;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordLengthCount {
    public static class WordLengthCountMapper extends Mapper<LongWritable, Text, Text,
IntWritable> {

        public void map(LongWritable key, Text value, Context context) throws IOException,
InterruptedException {
                String line = value.toString();

                StringTokenizer tokenizer = new StringTokenizer (line);

                while (tokenizer.hasMoreTokens()) {
                  String word = tokenizer.nextToken();
                  int length = word.length();
                  context.write(

                    new Text(String.valueOf(length)),

                    new IntWritable(1)
                  );
                }
        }
    }
```

```
    public static class WordLengthCountReducer extends Reducer <Text, IntWritable, Text, IntWritable >
{
        public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {
                int sum = 0;

                for (IntWritable x: values) {
                    sum += x.get();
                }
                context.write(key, new IntWritable(sum) );
        }

    }
    public static void main(String[] args) throws Exception{

        // TODO Auto-generated method stub
        Configuration conf = new Configuration();

        Job job = Job.getInstance(conf, "my count");
        job.setJarByClass(WordLengthCount.class);

        job.setMapperClass(WordLengthCountMapper.class);
        job.setReducerClass(WordLengthCountReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0: 1);
    }
}
```

5. **Perform log analysis of shared HDFS log file.**

Code:

```
package logAssignment;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import java.io.IOException;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class logAssignment {

   public static class LogMapper extends Mapper<Object, Text, Text, IntWritable> {

      private final static IntWritable one = new IntWritable(1);
      private final Text outKey = new Text();

      private static final Pattern IP_PATTERN =
            Pattern.compile("(\\d{1,3}\\.\\d{1,3}\\.\\d{1,3}\\.\\d{1,3}):50010");
      private static final Pattern BLOCK_PATTERN =
            Pattern.compile("(blk_-?\\d+)");

      @Override
      public void map(Object key, Text value, Context context) throws IOException, InterruptedException
{
         String line = value.toString().trim();
         if (line.isEmpty()) return;

         // date time pid LEVEL Component: message  (split into max 6 parts)
         String[] parts = line.split("\\s+", 6);
         if (parts.length < 6) return;

         String level = parts[3];                    // Feature 1: LEVEL
         String component = parts[4].replace(":", "");       // Feature 2: COMPONENT
         String message = parts[5];
         outKey.set("LEVEL~" + level);
         context.write(outKey, one);

         outKey.set("COMPONENT~" + component);
         context.write(outKey, one);
```

```
        String event = classifyEvent(message);
        if (event != null) {
            outKey.set("EVENT~" + event);
            context.write(outKey, one);
        }

        Matcher ipMatcher = IP_PATTERN.matcher(message);
        if (ipMatcher.find()) {
            outKey.set("DATANODE~" + ipMatcher.group(1));
            context.write(outKey, one);
        }

        Matcher blockMatcher = BLOCK_PATTERN.matcher(message);
        if (blockMatcher.find()) {
            outKey.set("BLOCK~" + blockMatcher.group(1));
            context.write(outKey, one);
        }
    }
    private String classifyEvent(String message) {
        if (message.contains("Receiving block")) return "ReceivingBlock";
        if (message.contains("Received block")) return "ReceivedBlock";
        if (message.contains("terminating")) return "PacketResponderTerminating";
        if (message.contains("addStoredBlock")) return "BlockMapUpdated";
        if (message.contains("allocateBlock")) return "AllocateBlock";
        if (message.contains("Verification succeeded")) return "VerificationSucceeded";
        if (message.contains("Deleting block")) return "DeletingBlock";
        return null;
    }
}

public static class LogReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    private final IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}

public static void main(String[] args) throws Exception {
    if (args.length < 2) {
        System.err.println("Usage: logAnalysis <input path> <output path>");
        System.exit(-1);
    }

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "HDFS Log Analysis - Main Features Only");
```

```
        job.setJarByClass(logAssignment.class);

        job.setMapperClass(LogMapper.class);
        job.setReducerClass(LogReducer.class);
        job.setCombinerClass(LogReducer.class); // safe: same op (sum) as reducer, cuts shuffle traffic

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```